مشكلة المخزن المحدود

🎯 الصورة الكاملة

المشكلة الكلاسيكية الأولى:

```text
Producer-Consumer Problem
= Bounded-Buffer Problem
```

السيناريو:

```text
Producer (المنتج): ينتج عناصر → يضعها في Buffer
Consumer (المستهلك): يسحب عناصر من Buffer → يستهلكها

التحديات:
1. Buffer محدود الحجم (n slots)
2. Producer لا يضيف إذا Buffer ممتلئ ❌
3. Consumer لا يسحب إذا Buffer فارغ ❌
4. لا Race Condition (Mutual Exclusion) ✅
```

📍 الجزء الأول: المفهوم الأساسي

1️⃣ فكرة المشكلة

🔑 التعريف

Bounded-Buffer Problem:

- **مشكلة كلاسيكية** في التزامن
- تُسمى أيضاً **Producer-Consumer Problem**
- **عمليتان** تتعاونان عبر **Buffer مشترك**

الأطراف:

1. Producer (المنتج):

- **ينتج** عناصر/بيانات
- **يضعها** في Buffer
- **ينتظر** إذا Buffer **ممتلئ**

2. Consumer (المستهلك):

- **يسحب** عناصر من Buffer
- **يستهلكها** (معالجة)
- **ينتظر** إذا Buffer **فارغ**

📊 مثال من الحياة

مثال McDonald's:

```text
الطباخ (Producer):
  - يطبخ برجر
  - يضعه في الصندوق (Buffer)
  - إذا الصندوق امتلأ → ينتظر ⏳

الزبون (Consumer):
  - يأخذ برجر من الصندوق
  - يأكله
  - إذا الصندوق فارغ → ينتظر ⏳

الصندوق = Bounded Buffer (حجم محدود)
```

🎯 التحديات

ثلاثة تحديات رئيسية:

1. Buffer Full:

```text
إذا Buffer ممتلئ (n/n):
  Producer يجب أن ينتظر ⏳
  لا يضيف ❌
```

2. Buffer Empty:

```text
إذا Buffer فارغ (n/0):
  Consumer يجب أن ينتظر ⏳
  لا يسحب ❌
```

3. Race Condition:

```text
إذا Producer و Consumer يصلان Buffer معاً:
- Data corruption ❌
- فوضى ❌

الحل: Mutual Exclusion ✅
```

2️⃣ الهيكل والسيمافورات

📊 البيانات المشتركة

1. Buffer:

```c
Item buffer[n];  // مصفوفة بحجم n
int in = 0;      // مؤشر الإضافة (Producer)
int out = 0;     // مؤشر السحب (Consumer)
```

الخصائص:

- **حجم ثابت** = n slots
- **Circular** (دائري) - يرجع للبداية عند النهاية
- **Critical Section**

🔧 السيمافورات الثلاث

الحل يستخدم 3 Semaphores:

1. mutex (Binary Semaphore):

```c
Semaphore mutex = 1;  // القيمة الابتدائية
```

الوظيفة:

- **Mutual Exclusion**
- يضمن أن **واحد فقط** يصل Buffer
- **Producer أو Consumer** - ليس كلاهما معاً

2. empty (Counting Semaphore):

```c
Semaphore empty = n;  // القيمة الابتدائية
```

الوظيفة:

- **يعد الأماكن الفارغة** في Buffer
- Producer يستخدمه: "كم مكان فارغ؟"
- عندما Buffer → **empty = 0** ممتلئ → Producer ينتظر

لماذا البداية = n؟

```text
Buffer يبدأ فارغاً تماماً
↓
كل الأماكن فارغة
↓
✅ empty = n
```

3. full (Counting Semaphore):

```c
Semaphore full = 0;  // القيمة الابتدائية
```

الوظيفة:

- **يعد الأماكن الممتلئة** في Buffer
- Consumer يستخدمه: "كم عنصر موجود؟"
- عندما Buffer → **full = 0** فارغ → Consumer ينتظر

لماذا البداية = 0؟

```text
Buffer يبدأ فارغاً
↓
لا عناصر موجودة
↓
✅ full = 0
```

📊 الجدول الشامل

السيمافور	النوع	القيمة الابتدائية	الوظيفة	من يستخدمه
mutex	Binary	1	Mutual Exclusion	**كلاهما**
empty	Counting	n	عدد الفارغة	**Producer** (wait)
full	Counting	0	عدد الممتلئة	**Consumer** (wait)

🔄 العلاقة بين **empty** و **full**

القاعدة الذهبية:

```text
empty + full = n (دائماً!)
```

الأمثلة:

```text
Buffer فارغ تماماً:
empty = n, full = 0
✅ n + 0 = n

Buffer ممتلئ تماماً:
empty = 0, full = n
✅ n = n + 0

Buffer نصف ممتلئ:
empty = n/2, full = n/2
✅ n/2 + n/2 = n
```

⭐ **المفهوم:**

```text
Producer: يستهلك "فارغة" → ينتج "ممتلئة"
wait(empty) ... signal(full)

Consumer: يستهلك "ممتلئة" → ينتج "فارغة"
wait(full) ... signal(empty)
```

📍 الجزء الثالث: Producer Process

1. "فيكة" مكان الإنتاج (`produce_item`)

- **القاعدة:** عملية صنع العنصر (`produce_item`) لازم تكون **خارج (Outside)** المنطقة الحرجة.
- **ليش؟ (سؤال MCQ):** حتى نزيد الكفاءة ونسمح للـ Consumer يشتغل بنفس الوقت. لو حطيناها داخل الـ CS، راح نقفل الباب لفترة طويلة ونعطل النظام كله.

2. فخ الـ Deadlock (ترتيب الـ `wait`)

هذا أهم جزء بالجابتر كله. لازم الترتيب يكون هيج:

1. `wait(empty)` : أحجز مكان فارغ أولاً.
2. `wait(mutex)` : أقفل الباب وادخل.

> ⚠️ **تنبيه للمتحانات:** لو عكستهم (يعني قفلت الباب قبل ما تتأكد أكو مكان)، والـ Buffer طلع مليان، الـ Producer راح ينام وهو **ماخذ المفتاح وياه**. المشتري (Consumer) ما راح يگدر يدخل يفرغ مكان، والنتيجة هي **Deadlock (استعصاء)**.

3. الـ Buffer الدائري (Circular Buffer)

- المعادلة المستخدمة هي: $in = (in + 1) \pmod n$
- فائدتها إنها ترجع المؤشر للبداية (الصفر) لما يوصل لنهاية المصفوفة، فيظل الـ Buffer يفتر بشكل دائري.

📝 "برشامة" التسلسل (احفظها هيج)

الخطوة	الأمر البرمجي	الوظيفة (للفهم)
قبل الدخول	`wait(empty)`	يسأل: "أكو مكان فارغ؟" (ينقص عداد الفراغ).
قفل الباب	`wait(mutex)`	يدخل للمنطقة الحرجة ويمنع أي تداخل.
الشغل (CS)	`buffer[in] = item`	يحط الغراض بالـ Buffer ويحرك المؤشر.
فتح الباب	`signal(mutex)`	يگول: "أني خلصت شغل، الرف هسة متاح".
الإشعار	`signal(full)`	يگول للـ Consumer: "تفضل، أكو غرض جديد جاهز".

📍 الجزء الرابع: Consumer Process

1. أولوية التحقق من المورد (`wait(full)`)

- **القاعدة:** المستهلك يبدأ دائماً بالـ `wait` على السيمافور الذي يمثل "العناصر الممتلئة" (`full`).
- **المنطق:** لا يمكن استهلاك شيء غير موجود. إذا كانت القيمة 0، يوضع المستهلك في حالة **Waiting** (يُعطل) حتى يقوم المنتج بزيادة هذا العداد.

2. فخ الاستعصاء (ترتيب الـ `wait` القاتل) ⚠️

- **التسلسل الإلزامي:** يجب تنفيذ `wait(full)` **قبل** `wait(mutex)` .
- **ليش؟ (سؤال امتحاني متكرر):** لو عكسنا الترتيب وقفلنا الـ `mutex` أولاً وكان المخزن فارغاً، المستهلك سينام وهو ممسك بالقفل. المنتج لن يستطيع الدخول ليضع عنصراً (لأن القفل مع المستهلك النائم)، والنتيجة هي **Deadlock**.

3. تحرير الفراغات للمنتج (`signal(empty)`)

- **الآلية:** بعد الخروج من المنطقة الحرجة، المستهلك يرسل إشارة `signal(empty)` .
- **الوظيفة:** هو بهذا الفعل يُعلم المنتج بأن "مكاناً قد فُرِغ"، فإذا كان المنتج نائماً بسبب امتلاء المخزن، سيتم إيقاظه الآن.

4. موقع المعالجة (`consume_item`)

- **القاعدة:** معالجة البيانات أو استهلاكها تتم **خارج (Outside)** المنطقة الحرجة.
- **السبب:** لتقليل الوقت الذي يقضيه الخيط داخل الـ CS، مما يسمح للمنتج بالدخول والعمل بالتوازي (Maximize Throughput).

📝 **"برشامة" تسلسل المستهلك (للمراجعة السريعة)**

الخطوة	الأمر البرمجي	التأثير التقني
1. الحجز	`wait(full)`	ينقص عداد العناصر الممتلئة (يتحقق من الوجود).
2. القفل	`wait(mutex)`	يمنع أي خيط آخر من الوصول للـ Buffer.
3. العمل	`item = buffer[out]`	يسحب البيانات ويحدث المؤشر `out = (out+1)%n` .
4. الفتح	`signal(mutex)`	يحرر الوصول للـ Buffer.
5. الإشعار	`signal(empty)`	يزيد عداد الفراغات (يُعطي فرصة للمنتج).

📍 الجزء الخامس: التنسيق والتحليل

5️⃣ التناغم بين Producer و Consumer

🔄 العلاقة التكاملية

Producer ينتج لـ Consumer:

```text
Producer:
  - يستهلك: empty (أماكن فارغة)
  - ينتج: full (أماكن ممتلئة)

wait(empty) ... signal(full)
```

Consumer ينتج لـ Producer:

```text
Consumer:
  - يستهلك: full (أماكن ممتلئة)
  - ينتج: empty (أماكن فارغة)

wait(full) ... signal(empty)
```

🔒 ضمانات الحل

1. Mutual Exclusion: ✅

```text
mutex يضمن:
  - واحد فقط في CS
  - Producer أو Consumer
  - لا تعارض ✅
```

2. No Buffer Overflow: ✅

```text
empty يضمن:
  - Producer لا يضيف إذا ممتلئ
  - wait(empty) → ينام عند 0
```

3. No Empty Buffer Read: ✅

```text
full يضمن:
  - Consumer لا يسحب إذا فارغ
  - wait(full) → ينام عند 0
```

4. Progress: ✅

```text
- إذا Buffer فارغ → Producer يعمل
- إذا Buffer ممتلئ → Consumer يعمل
- لا deadlock ✅
```

5. Bounded Waiting: ✅

```text
- FIFO queue في Semaphore
- عادل ✅
```

⚠️ خطأ شائع: الترتيب الخاطئ

❌ خطأ فادح:

Producer الخاطئ:

```c
wait(mutex);   // خذ القفل أولاً ❌
wait(empty);   // تم انتظر فارغة
// المشكلة: إذا Buffer ممتلئ...
```

السيناريو الكارثي:

```text
1. Producer: wait(mutex) ✅ (أخذ القفل)
2. Producer: wait(empty) ⏳ (Buffer ممتلئ - ينام!)
   → لكن القفل معه! ❌
3. Consumer: wait(full) ✅ (يوجد عنصر)
4. Consumer: wait(mutex) ⏳ (القفل مع Producer النائم!)
   → Consumer ينام ❌

النتيجة:
  Producer: نائم وينتظر Consumer
  Consumer: نائم وينتظر Producer
  = Deadlock! ❌❌❌
```

✅ الترتيب الصحيح:

```text
wait(empty/full) ← أولاً (قبل القفل)
wait(mutex)      ← ثانياً
```

⭐ **للحفظ:** أبداً wait(mutex) قبل wait(empty/full) ❌

📊 المقارنة الكاملة

الجانب	Producer	Consumer
يسأل عن	**empty** (فارغة؟)	**full** (ممتلئة؟)
الترتيب	wait(empty) → wait(mutex)	wait(full) → wait(mutex)
في CS	**add** to buffer	**remove** from buffer
الخروج	signal(mutex) → signal(full)	signal(mutex) → signal(empty)
ينتج	**full** (للـ Consumer)	**empty** (للـ Producer)
ينام عندما	empty = 0 (ممتلئ)	full = 0 (فارغ)

🔥 ورقة الغش النهائية

الخطأ الشائع

```text
❌ أبداً:
wait(mutex);  // أولاً
wait(empty);  // ثانياً
= Deadlock!

✅ الصحيح:
wait(empty);  // أولاً
wait(mutex);  // ثانياً
```

المفهوم

```text
Producer → Buffer → Consumer
   ينتج       (n)       يستهلك

التحديات:
  - Buffer ممتلئ → Producer ينتظر
  - Buffer فارغ → Consumer ينتظر
  - Race Condition → Mutex
```

السيمافورات الثلاث

```text
mutex = 1   (Binary)
  - Mutual Exclusion
  - واحد فقط في CS

empty = n   (Counting)
  - عدد الأماكن الفارغة
  - Producer يستخدمه

full = 0    (Counting)
  - عدد الأماكن الممتلئة
  - Consumer يستخدمه

القاعدة: empty + full = n
```

التكامل

```text
Producer: يستهلك empty → ينتج full
Consumer: يستهلك full → ينتج empty

كل واحد ينتج ما يستهلكه الآخر! ✅
```

المزامنة داخل النواة

🎯 الصورة الكاملة

لماذا المزامنة داخل النواة؟

```text
النواة (Kernel):
  - تدير موارد النظام
  - بيانات هيكلية مشتركة (Kernel Data Structures)
  - عمليات متوازية (Concurrent Processes/Threads)

المشكلة:
  - Race Conditions داخل النواة ❌
  - تلف بيانات النظام ❌
  - انهيار كامل (Crash) ❌

الحل:
  - آليات مزامنة خاصة بالنواة ✅
```

الأنظمة المدروسة:

1. **Windows** - Dispatcher Objects
2. **Linux** - Atomic, Mutex, Spinlocks

📍 الجزء الأول: Windows Synchronization

1️⃣ مقدمة Windows Kernel

🔑 استراتيجية Windows

تختلف حسب:

- **نوع المعالج:** Single vs Multiprocessor
- **المكان:** Kernel Space vs User Space
- **المدة:** Short vs Long duration

2️⃣ داخل النواة: Kernel Space

📊 Single-Processor Systems

الآلية:

```text
Interrupt Masking (إيقاف المقاطعات)
```

كيف تعمل:

```c
// حماية مورد عالمي
disable_interrupts();  // أوقف المقاطعات ⛔

// Critical Section
// ... الوصول للمورد العالمي ...

enable_interrupts();   // أعد المقاطعات ✅
```

الفكرة:

- **توقيف المقاطعات** مؤقتاً
- **لا** context switch ✅
- العملية الحالية **لا تُقاطع**
- **بسيط وفعال** على Single CPU

📊 Multiprocessor Systems

الآلية:

```text
Spinlocks (أقفال الدوران)
```

الخصائص:

- **للكود القصير جداً فقط** ✅
- **لا** Preemption أثناء حمل Spinlock
- **عالي الكفاءة** للعمليات السريعة

القاعدة المهمة:

```text
Thread يحمل Spinlock:
- ممنوع Preemption ❌
- يجب أن ينهي بسرعة
- لا context switch
```

3️⃣ خارج النواة: Thread Synchronization

🔧 Dispatcher Objects

التعريف:

- **أدوات مزامنة** على مستوى Threads
- توفرها النواة للتطبيقات

الأنواع:

```text
1. Mutex Objects
2. Semaphore Objects
3. Event Objects
4. Timer Objects
5. Thread Objects
```

🔴🟢 المفهوم الجوهري: Signaled vs Nonsignaled

كل Dispatcher Object له حالة:

1. Signaled State (الإشارة الخضراء 🟢):

```text
المعنى: المورد متاح ✅
النتيجة: Thread لا يتوقف - يأخذ المورد مباشرة
مثال: القفل مفتوح، Semaphore > 0
```

2. Nonsignaled State (الإشارة الحمراء 🔴):

```text
المعنى: المورد غير متاح ❌
النتيجة: Thread يدخل Waiting State ⏳
مثال: القفل مقفول، Semaphore = 0
```

📝 كيف تعمل؟

دوال الانتظار:

```c
KeWaitForSingleObject()      // انتظر كائن واحد
KeWaitForMultipleObjects()   // انتظر عدة كائنات
```

الآلية:

```text
1. Thread يستدعي KeWaitForSingleObject(object)

2. النواة تفحص حالة الكائن:

   إذا Signaled (🟢):
     - Thread يأخذ المورد ✅
     - يستمر في التنفيذ

   إذا Nonsignaled (🔴):
     - Thread → Waiting State 😴
     - يُضاف لطابور الانتظار
     - يُزال من CPU

3. عندما كائن آخر يُشير (signal):
     - الكائن → Signaled
     - Threads في الطابور → Ready State 🔔
     - Scheduler يختار أحدهم للتنفيذ
```

🔧 أنواع Events

Notification Event (Manual-Reset):

```text
عندما يُشار (Signaled):
  - يبقى Signaled ✅
  - كل Threads في الانتظار تُطلق ✅
  - يجب reset يدوياً
```

Synchronization Event (Auto-Reset):

```text
عندما يُشار (Signaled):
  - Thread واحد فقط يُطلق ✅
  - الكائن يرجع Nonsignaled تلقائياً
```

4️⃣ Critical-Section Objects

🔑 أداة ذكية

الخصائص:

- تعمل في **User-mode** (لا Kernel call)
- **أسرع** من Kernel objects
- خاصة بـ **Multiprocessor**

الآلية الذكية:

```text
في Multiprocessor:
  1. Thread يحاول أخذ القفل
  2. إذا مقفول → Spinlock قليلاً 🔄
     (يدور عدة مرات)
  3. إذا لم ينفتح → System Call للنواة
     (ينام في الطابور 😴)
```

الفائدة:

```text
غالباً القفل ينفتح بسرعة
↓
Spinlock ينجح قبل System Call
↓
توفير تكلفة Kernel call ✅
↓
أداء عالي جداً! ✅
```

📊 ملخص Windows

الحالة	الآلية	متى تُستخدم
Single CPU - Kernel	Interrupt Masking	حماية موارد عالمية
Multi CPU - Kernel	Spinlocks	كود قصير جداً
Thread Sync	Dispatcher Objects	Mutex, Semaphore, Events
User-mode	Critical-Section	أداء عالي

📍 الجزء الثاني: Linux Synchronization

5️⃣ مقدمة Linux Kernel

🔑 التطور المهم

Linux 2.6 وما بعد:

```text
✅ Fully Preemptive Kernel
```

المعنى:

- **أي مهمة** داخل النواة **يمكن إيقافها**
- مهمة **أعلى أولوية** تستطيع مقاطعة
- **المزامنة أصبحت ضرورية** جداً

⭐ **للحفظ:** Linux 2.6+ = **Fully Preemptive** - يمكن إيقاف أي مهمة

6️⃣ الآلية الأولى: Atomic Integers

🔑 التعريف

Atomic Integers:

- **أبسط** آلية مزامنة
- **للعمليات الحسابية البسيطة** فقط
- **بدون locks** ✅

نوع البيانات:

```c
atomic_t counter;  // نوع خاص
```

الخاصية:

```text
كل عملية = Uninterruptible ✅
لا يمكن مقاطعتها
Atomic = غير قابلة للتجزئة
```

✅ المميزات

1. لا Overhead:

```text
بدون Mutex:
  mutex_lock()
  count++
  mutex_unlock()
  → بطيء ❌

مع Atomic:
  atomic_inc(&count)
  → سريع جداً ✅
```

2. بسيط:

- سطر واحد
- لا تعقيد

3. آمن:

- مضمون atomic
- لا race condition

7️⃣ الآلية الثانية: Mutex Locks

🔑 التعريف

Mutex في Linux Kernel:

- **لحماية Critical Sections** داخل النواة
- إذا غير متاح → Thread **ينام** 😴

الاستخدام:

```c
struct mutex my_mutex;
mutex_init(&my_mutex);

// الحماية
mutex_lock(&my_mutex);
// Critical Section
mutex_unlock(&my_mutex);
```

📊 الخصائص

عندما Mutex مقفول:

```text
Thread يستدعي mutex_lock():
  1. Mutex مقفول ❌
  2. Thread → Sleep State 😴
  3. يُزال من CPU
  4. Reschedule يحدث ✅

عندما Mutex يفتح:
  5. Thread يستيقظ 🔔
  6. يأخذ Mutex ويدخل CS
```

⭐ الفرق عن Spinlock:

```text
Mutex:
  - ينام عند الانتظار 😴
  - يمكن استخدامه في Interrupts ❌
  - للـ CS طويلة ✅

Spinlock:
  - يدور عند الانتظار 🔄
  - لا ينام
  - للـ CS قصيرة جداً ✅
```

8️⃣ الآلية الثالثة: Spinlocks

🔑 التعريف

Spinlock في Linux:

- **للأجهزة متعددة المعالجات (SMP)**
- **للفترات القصيرة جداً** فقط
- **لا نوم** - دوران مستمر

الاستخدام:

```c
spinlock_t my_lock;
spin_lock_init(&my_lock);

spin_lock(&my_lock);
// Critical Section (قصيرة جداً!)
spin_unlock(&my_lock);
```

📊 كيف تعمل؟

Multiprocessor (SMP):

```text
CPU1: spin_lock(&lock) ✅ (أخذه)
      // CS

CPU2: spin_lock(&lock) 🔄 (يدور!)
      while (lock == taken) {
          // busy wait - يستهلك CPU
      }

CPU1: spin_unlock(&lock) ✅

CPU2: (الآن أخذه!) ✅
```

```text
Thread يحمل Spinlock:
- Preemption مُعطل ❌
- لا يمكن إيقافه
- يجب أن ينهي بسرعة
```

📊 Single-Processor Systems

المشكلة:

```text
على Single CPU:
- Thread يدور في Spinlock 🔄
- لكن لا CPU آخر لفتح القفل! ❌
- Deadlock! ❌
```

الحل الذكي:

```c
// على Single CPU، Linux يستبدل:
spin_lock(&lock) → preempt_disable()
spin_unlock(&lock) → preempt_enable()

لا Spinlock حقيقي!
فقط تعطيل Preemption ✅
```

⭐ **للحفظ:**

- **Multi-CPU:** Spinlock حقيقي
- **Single-CPU:** preempt_disable فقط

⚠️ تحذيرات مهمة

لا تفعل هذا:

```c
spin_lock(&lock);
sleep();              // ❌❌❌ أبداً!
mutex_lock();         // ❌❌❌ أبداً!
schedule();           // ❌❌❌ أبداً!
spin_unlock(&lock);
```

لماذا؟

```text
Spinlock يعطل Preemption
↓
إذا نمت أو انتظرت Mutex
↓
لا يمكن للنظام جدولة threads أخرى
↓
❌ Deadlock!
```

القواعد:

```text
أثناء حمل Spinlock:
1. لا نوم ❌
2. لا mutex_lock ❌
3. لا أي شيء يسبب reschedule ❌
4. اخرج بسرعة! ✅
```

9️⃣ الآلية الرابعة: Preemption Management

🔑 المفهوم

Linux ذكي في التعامل مع Preemption:

كل Task لها:

```c
int preempt_count;  // عداد
```

القاعدة:

```text
preempt_count > 0:
  → Task غير آمن إيقاف ❌
  → Preemption معطل

preempt_count = 0:
  → Task آمن إيقاف ✅
  → Preemption ممكن
```

📊 المثال

```text
                                        :Task A
                  preempt_count = 0  ← يمكن إيقافها ✅

                    spin_lock(&lock1)
                  preempt_count = 1  ← لا يمكن إيقافها ❌

                   spin_lock(&lock2)
                  preempt_count = 2  ← لا يمكن إيقافها ❌

                  spin_unlock(&lock2)
                  preempt_count = 1  ← لا يزال غير آمن ❌

                   spin_unlock(&lock1)
                  preempt_count = 0  ← الآن آمن ✅
```

⭐ **للحفظ:** preempt_count = **عداد الأقفال** - صفر = آمن للإيقاف

📊 ملخص Linux

الآلية	الاستخدام	ينام؟	للمدة
Atomic Integers	عدادات بسيطة	لا	-
Mutex	CS داخل النواة	نعم 😴	طويلة/متوسطة
Spinlock	SMP - CS قصيرة	لا 🔄	قصيرة جداً
Preemption Mgmt	تتبع الأمان	-	-

Preemptive Kernel:

- ✅ Linux 2.6+ = **Fully Preemptive**
- preempt_count يتحكم في الأمان

📍 الجزء الثالث: المقارنة

🔟 Windows vs Linux

📊 الفلسفة

الجانب	Windows	Linux
المفهوم الأساسي	Dispatcher Objects + States	متعدد الآليات
الحالات	Signaled/Nonsignaled	لا يوجد
User-mode	Critical-Section (ذكية)	لا أدوات خاصة
Kernel Preemption	جزئي	Fully Preemptive
الأبسط	Mutex/Semaphore Objects	Atomic Integers

📊 Single vs Multi Processor

النوع	Windows	Linux
Single CPU	Interrupt Masking	preempt_disable
Multi CPU	Spinlocks	Spinlocks
Thread Sync	Dispatcher Objects	Mutex/Semaphore

Sleep vs Spin 📊

Windows:

```text
Dispatcher Object Nonsignaled:
  → Thread ينام 😴
  → ينتظر Signaled

Spinlock (Kernel):
  → للكود القصير
  → لا Preemption
```

Linux:

```text
Mutex locked:
  → Thread ينام 😴
  → Reschedule يحدث

Spinlock:
  → يدور 🔄
  → Preemption معطل
  → للكود القصير جداً
```

🔥 ورقة الغش النهائية

Windows

```text
Single CPU:
  - Interrupt Masking

Multi CPU:
  - Spinlocks (كود قصير)

Dispatcher Objects (5):
  - Mutex, Semaphore, Event, Timer, Thread

States:
  - Signaled (🟢): متاح - لا انتظار
  - Nonsignaled (🔴): غير متاح - انتظر

User-mode:
  - Critical-Section Objects
  - Spinlock تم Sleep (ذكية)
```

```text
Fully Preemptive Kernel (2.6+)

Atomic Integers:
- atomic_t
- للعدادات البسيطة
- أسرع من Mutex

Mutex:
- ينام عند الانتظار 😴
- Reschedule يحدث
- لا في Interrupt context ❌

Spinlock:
- يدور عند الانتظار 🔄
- preempt_disable()
- للكود القصير جداً
- لا نوم أثناءه ❌

Single CPU: preempt_disable فقط
Multi CPU: Spinlock حقيقي

Preemption Management:
- preempt_count
- > 0 = غير آمن إيقاف
- = 0 = آمن إيقاف
```